

Все Магические (dunder) методы в Python

Конструктор, инициализатор и финализатор

Operation	Dunder Method Call	Returns
T(a, b=3)	T.__new__(T, a, b=3)	New instance (x)
T(a, b=3)	T.__init__(x, a, b=3)	None
del x	x.__del__()	None

Вызов класса возвращает новый экземпляр класса благодаря методу `__new__`. Метод `__new__` - это метод конструктора в Python, хотя, в отличие от конструкторов во многих языках программирования, вам почти никогда не следует переопределять метод `__new__`. Для управления созданием объекта предпочтительнее использовать инициализатор `__init__`, а не конструктор `__new__`. Вот странный пример `__new__`.

Вы могли бы подумать о методе `__del__` как о методе "деструктора", хотя на самом деле он называется методом завершения. Непосредственно перед удалением объекта вызывается его метод `__del__`. Файлы реализуют метод `__del__`, который закрывает файл и любой буфер двоичного файла, с которым он может быть связан.

Равенство и хэшируемость

Operation	Dunder Method Call	Returns
x == y	x.__eq__(y)	Typically bool
x != y	x.__ne__(y)	Typically bool
hash(x)	x.__hash__()	int

Метод `__eq__` в Python обычно возвращает значение True, False. Реализация `__eq__` по умолчанию основана на операторе is, который проверяет идентичность.

Реализация `__ne__` по умолчанию вызывает `__eq__` и отменяет любое возвращаемое логическое значение.

Хэшируемые объекты могут использоваться в качестве ключей в словарях или в множествах. По умолчанию все объекты вашего класса хэшируемые, но если вы переопределили метод `__eq__`, то ваши объекты не будут доступны для хэширования без переопределения метода `__hash__`.

Упорядочиваемость

Operation	Dunder Method Call	Returns
<	__lt__	Typically bool
>	__gt__	Typically bool
<=	__le__	Typically bool
>=	__ge__	Typically bool

Преобразование типов и форматирование строк

Function	Dunder Method Call	Returns
<code>str(x)</code>	<code>x.__str__()</code>	<code>str</code>
<code>bool(x)</code>	<code>x.__bool__()</code>	<code>bool</code>
<code>int(x)</code>	<code>x.__int__()</code>	<code>int</code>
<code>float(x)</code>	<code>x.__float__()</code>	<code>float</code>
<code>bytes(x)</code>	<code>x.__bytes__()</code>	<code>bytes</code>
<code>complex(x)</code>	<code>x.__complex__()</code>	<code>complex</code>
<code>f"{x:s}"</code>	<code>x.__format__(s)</code>	<code>str</code>
<code>repr(x)</code>	<code>x.__repr__()</code>	<code>str</code>

Контекстные менеджеры

Use	Dunder Method Call	Returns
<code>with</code> block enter	<code>x.__enter__()</code>	A value given to <code>as</code>
<code>with</code> block exit	<code>x.__exit__(exc_type, exc, traceback)</code>	Truthy/falsey value

Контейнеры и коллекции

Operation	Dunder Method Call	Return Type	Implemented
<code>len(x)</code>	<code>x.__len__()</code>	integer	Very common
<code>iter(x)</code>	<code>x.__iter__()</code>	iterator	Very common
<code>for item in x: ...</code>	<code>x.__iter__()</code>	iterator	Very common
<code>x[a]</code>	<code>x.__getitem__(a)</code>	any object	Common
<code>x[a] = b</code>	<code>x.__setitem__(a, b)</code>	None	Common
<code>del x[a]</code>	<code>x.__delitem__(a)</code>	None	Common
<code>a in x</code>	<code>x.__contains__(a)</code>	bool	Common
<code>reversed(x)</code>	<code>x.__reversed__()</code>	iterator	Common
<code>next(x)</code>	<code>x.__next__()</code>	any object	Uncommon
<code>x[a]</code>	<code>x.__missing__(a)</code>	any object	Uncommon
<code>operator.length_hint(x)</code>	<code>x.__length_hint__()</code>	integer	Uncommon

Метод `__iter__` используется функцией `iter` и всеми формами итерации: циклами `for`, списками, распаковкой кортежей и использованием `*` для распаковки итерируемых объектов

Вызываемость

Operation	Dunder Method Call	Return Type
x(a, b=c)	x.__call__(a, b=c)	any object

При вызове класса используется метод `__call__` его метакласса.
При вызове экземпляра класса используется метод `__call__` класса.

Арифметические операторы

математические арифметические операторы

Operation	Left-Hand Method	Right-Hand Method	Description
x + y	__add__	__radd__	Add / Concatenate
x - y	__sub__	__rsub__	Subtract
x * y	__mul__	__rmul__	Multiply
x / y	__truediv__	__rtruediv__	Divide
%	__mod__	__rmod__	Modulo
x // y	__floordiv__	__rfloordiv__	Integer division
**	__pow__	__rpow__	Exponentiate
x @ y	__matmul__	__rmatmul__	Matrix multiply

побитовые арифметические операторы

Operation	Left-Hand Method	Right-Hand Method	Description
x & y	__and__	__rand__	AND
x y	__or__	__ror__	OR
x ^ y	__xor__	__rxor__	XOR
x >> y	__rshift__	__rrshift__	Right-shift
x << y	__lshift__	__rlshift__	Left-shift

унарные арифметические операторы

Operation	Dunder Method	Variety	Description
-x	__neg__	Mathematical	Negate
+x	__pos__	Bitwise	Affirm
~x	__invert__	Bitwise	Invert

быстрые арифметические операторы

Operation	Dunder Method Call	Returns
x += y	x.__iadd__(y)	Typically self
x -= y	x.__isub__(y)	Typically self
x *= y	x.__imul__(y)	Typically self
x /= y	x.__itruediv__(y)	Typically self
x %= y	x.__imod__(y)	Typically self
x //= y	x.__ifloordiv__(y)	Typically self
x **= y	x.__ipow__(y)	Typically self
x @= y	x.__imatmul__(y)	Typically self
x &= y	x.__iand__(y)	Typically self
x = y	x.__ior__(y)	Typically self
x ^= y	x.__ixor__(y)	Typically self
x >>= y	x.__irshift__(y)	Typically self
x <<= y	x.__ilshift__(y)	Typically self

Встроенные математические функции

Operation	Dunder Method Call	Returns
divmod(x, y)	x.__divmod__(y)	2-item tuple
divmod(x, y)	y.__rdivmod__(x)	2-item tuple
abs(x)	x.__abs__()	float
sequence[x]	x.__index__()	int
round(x)	x.__round__()	Number
math.trunc(x)	x.__trunc__()	Number
math.floor(x)	x.__floor__()	Number
math.ceil(x)	x.__ceil__()	Number

Доступ к атрибутам

Operation	Dunder Method Call	Returns
<code>x.missing</code>	<code>x.__getattr__("missing")</code>	Attribute value
<code>x.anything</code>	<code>x.__getattribute__("anything")</code>	Attribute value
<code>x.thing = value</code>	<code>x.__setattr__("thing", value)</code>	<code>None</code>
<code>del x.thing</code>	<code>x.__delattr__("thing")</code>	<code>None</code>
<code>dir(x)</code>	<code>x.__dir__()</code>	List of strings

Метод `__getattribute__` вызывается для каждого доступа к атрибуту, в то время как метод `__getattr__` вызывается только после того, как Python не удастся найти данный атрибут.

Все вызовы методов и обращения к атрибутам вызывают `__getattribute__`, поэтому правильная реализация этого метода является сложной задачей.

Метод `__dir__` должен возвращать последовательность имен атрибутов (в виде строк). Когда функция `dir` вызывает `__dir__`, она преобразует последовательность в отсортированный список (как это делает `sorted`).

Встроенные функции `getattr`, `setattr` и `delattr` соответствуют одноименным методам dunder, но они предназначены только для динамического доступа к атрибутам (не для доступа ко всем атрибутам).

Метапрограммирование

Implemented on	Operation	Dunder Method Call	Returns
Metaclasses	<code>class T: ...</code>	<code>type(base).__prepare__()</code>	mapping
Metaclasses	<code>isinstance(x, T)</code>	<code>T.__instancecheck__(x)</code>	<code>bool</code>
Metaclasses	<code>issubclass(U, T)</code>	<code>T.__subclasscheck__(U)</code>	<code>bool</code>
Any class	<code>class U(T): ...</code>	<code>T.__init_subclass__(U)</code>	<code>None</code>
Any class	(Called manually)	<code>T.__subclasses__()</code>	<code>list</code>
Any class	<code>class U(x): ...</code>	<code>x.__mro_entries__([x])</code>	<code>tuple</code>
Any class	<code>T[y]</code>	<code>T.__class_getitem__(y)</code>	an item

Дескрипторы

Operation	Dunder Method Call	Returns
<code>class T: x = U()</code>	<code>T.x.__set_name__(T, 'x')</code>	<code>None</code>
<code>t.x</code>	<code>T.x.__get__(t, T)</code>	The value
<code>t.x = y</code>	<code>T.x.__set__(t, y)</code>	<code>None</code>
<code>del t.x</code>	<code>T.x.__delete__(t)</code>	<code>None</code>

Подробнее: https://proproprogs.ru/python_oop/deskriptyory-data-descriptor-i-non-data-descriptor

Буферы

Operation	Dunder Method Call	Returns
memoryview(x)	x.__buffer__(flags)	memoryview
del memoryview(x)	x.__release_buffer__(m)	None

Обычно реализуется на C, поскольку он предназначен для объектов низкого уровня.

Асинхронные операции

__aenter__: точно так же, как **__enter__**, но возвращает ожидаемый объект

__aexit__: точно так же, как **__exit__**, но возвращает ожидаемый объект

__aiter__: должен возвращать асинхронный итератор

__anext__: как **__next__** или несинхронные итераторы, но это должно возвращать ожидаемый объект, и это должно вызывать **StopAsyncliteration** вместо **Stopliteration**

__await__: возвращает итератор